E-ticaret Veri Ambarı

dbt ve BigQuery ile test edilen, belgelenmiş, katmanlı bir veri modeli

Modül 2 çalışması: dbt · BigQuery · Google Cloud Shell · Git

Hazırlayan: Zeynep

Tarih: Ekim 2026

Araçlar: dbt Core 1.12.5 · dbt-bigquery 1.12.1 · Google BigQuery · Cloud Shell

Veri: bigquery-public-data.thelook_ecommerce (sentetik moda e-ticaret verisi)

Özet

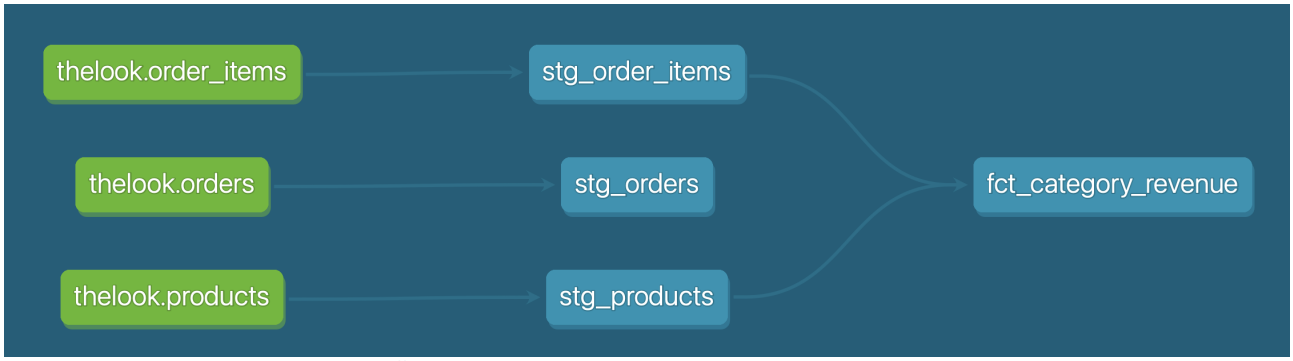
- Ham e-ticaret verisini **source → staging → mart** katmanlarına ayıran bir dbt projesi kuruldu.
- 3 staging modeli (view) ve 1 mart modeli (table) oluşturuldu. Modeller **13 veri kalitesi testiyle** korunuyor.
- Tüm proje tek komutla (**dbt build**) yaklaşık 14 saniyede kuruluyor: **17/17 adım başarılı**, hata ve uyarı yok.
- Mart tablosu, Modül 1'de iki ayrı sorguyla hesaplanan kategori bazlı net geliri tek, test edilmiş bir tabloda sunuyor.

1. Neden bir veri ambarı?

Modül 1'deki analizlerde aynı iş kuralı ("iptal ve iade edilen kalemler gelire sayılmaz") her sorguda yeniden yazıldı. Kategori bazlı net gelir için iki ayrı sorgu çalıştırılıp sonuçlar elle birleştirildi. Bu yaklaşım hataya açıktır: kuralın bir sorguda unutulması, raporlar arasında tutarsız rakamlar demektir.

dbt ile iş kuralları bir kez, tek bir yerde tanımlanıyor. Her tablo otomatik testlerle kontrol ediliyor ve tablolar arasındaki bağımlılıklar belgeleniyor. Dashboard'lar ve analizler artık ham veriye değil, test edilmiş mart tablolarına bağlanabilir.

2. Mimari



dbt docs ile üretilen lineage grafiği. Yeşil: ham kaynak tablolar, mavi: dbt modelleri.

Katman	Model	Tür	Amaç
Source	thelook.orders, order_items, products	–	Google'ın herkese açık ham verisi, değiştirilmez
Staging	stg_orders	view	Siparişler: gerekli kolonlar seçildi
Staging	stg_order_items	view	Sipariş kalemleri: id → order_item_id olarak adlandırıldı
Staging	stg_products	view	Ürünler: id → product_id olarak adlandırıldı
Mart	fct_category_revenue	table	Kategori bazında brüt, iptal, iade ve net gelir

Tasarım kararları: Staging modelleri **view** olarak tutuldu. Hafiftirler ve her sorguda kaynağın güncel halini okurlar. Mart modeli **table** olarak kaydedildi, çünkü dashboard'lar bu tabloyu sık okuyacak. Modeller birbirine **ref()**, kaynaklara **source()** ile bağlandı. Böylece dbt çalışma sırasını kendisi belirliyor.

3. Veri kalitesi testleri

Her modelin anahtar kolonları otomatik testlerle kontrol ediliyor. Bir test başarısız olursa hatalı veri dashboard'a ulaşmadan fark ediliyor.

Model	Kolon	Test	Sonuç
stg_orders	order_id	unique, not_null	**PASS**
stg_orders	status	accepted_values (5 durum)	**PASS**
stg_order_items	order_item_id	unique, not_null	**PASS**
stg_order_items	order_id	not_null	**PASS**
stg_order_items	product_id	not_null	**PASS**
stg_products	product_id	unique, not_null	**PASS**
stg_products	department	accepted_values (Men, Women)	**PASS**
fct_category_revenue	category	unique, not_null	**PASS**
fct_category_revenue	net_revenue	not_null	**PASS**

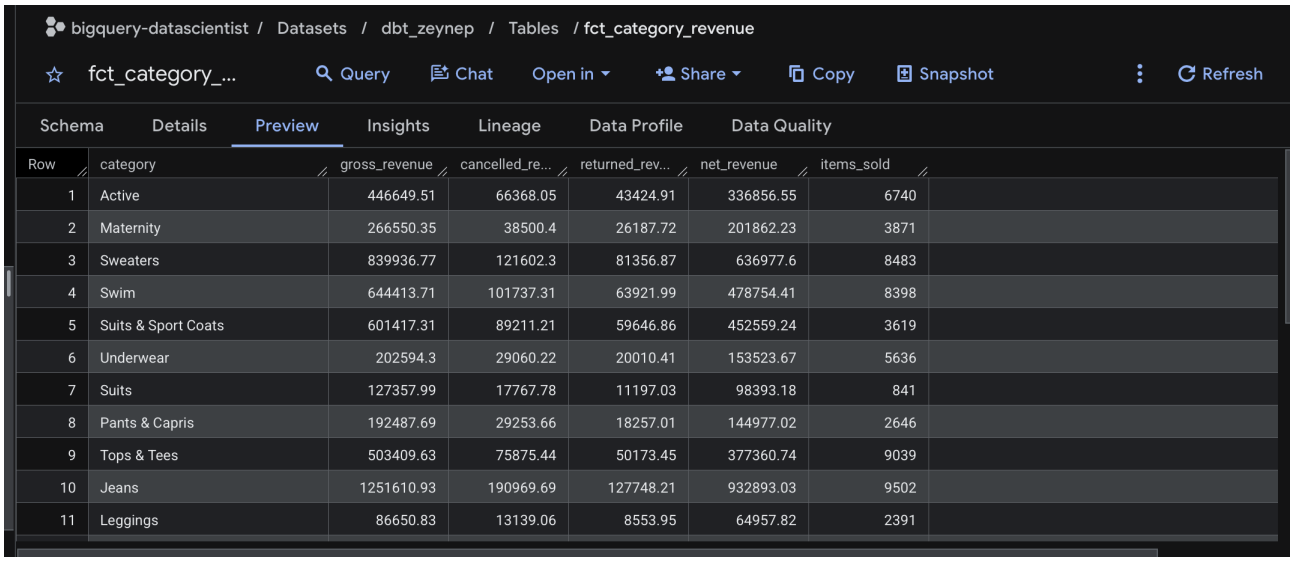
Test seçiminde dikkat edilen nokta: unique testi yalnızca tablonun "bir satır = bir kayıt" anahtarına konuldu. Örneğin sipariş kalemleri tablosunda bir siparişin birden fazla kalemi olduğu için order_id tekrar eder. Orada unique yerine sadece not_null kullanıldı, benzersiz anahtar order_item_id oldu.

dbt build sonucu

```
Found 4 models, 13 data tests, 3 sources
Concurrency: 4 threads (target='dev')
...
14 of 17 OK created sql table model dbt_zeynep.fct_category_revenue
         [CREATE TABLE (26.0 rows, 6.5 MiB processed) in 2.71s]
...
Finished running 1 table model, 13 data tests, 3 view models in 14.21s.
Completed successfully
Done. PASS=17 WARN=0 ERROR=0 SKIP=0 TOTAL=17
```

4. Mart tablosu: fct_category_revenue

Tablo 26 kategori için brüt geliri, iptal ve iade edilen tutarları, net geliri ve satılan kalem sayısını içeriyor. Aşağıda BigQuery'deki tablonun önizlemesi var:



Row	category	gross_revenue	cancelled_re...	returned_rev...	net_revenue	items_sold
1	Active	446649.51	66368.05	43424.91	336856.55	6740
2	Maternity	266550.35	38500.4	26187.72	201862.23	3871
3	Sweaters	839936.77	121602.3	81356.87	636977.6	8483
4	Swim	644413.71	101737.31	63921.99	478754.41	8398
5	Suits & Sport Coats	601417.31	89211.21	59646.86	452559.24	3619
6	Underwear	202594.3	29060.22	20010.41	153523.67	5636
7	Suits	127357.99	17767.78	11197.03	98393.18	841
8	Pants & Capris	192487.69	29253.66	18257.01	144977.02	2646
9	Tops & Tees	503409.63	75875.44	50173.45	377360.74	9039
10	Jeans	1251610.93	190969.69	127748.21	932893.03	9502
11	Leggings	86650.83	13139.06	8553.95	64957.82	2391

BigQuery'de oluşturulan tablonun önizlemesi (ilk 11 satır).

Doğrulama ve gözlemler

- **İç tutarlılık:** Her kategoride brüt − iptal − iade = net geliri tutuyor. Örneğin Active: 446.650 − 66.368 − 43.425 = 336.857 $.
- **Kalem başına ortalama fiyat kategoriler arasında çok farklı:**

Kategori	Net gelir ($)	Satılan kalem	Kalem başına ($)
Suits & Sport Coats	452.559	3.619	125,05
Jeans	932.893	9.502	98,18
Sweaters	636.978	8.483	75,09
Active	336.857	6.740	49,98
Tops & Tees	377.361	9.039	41,75

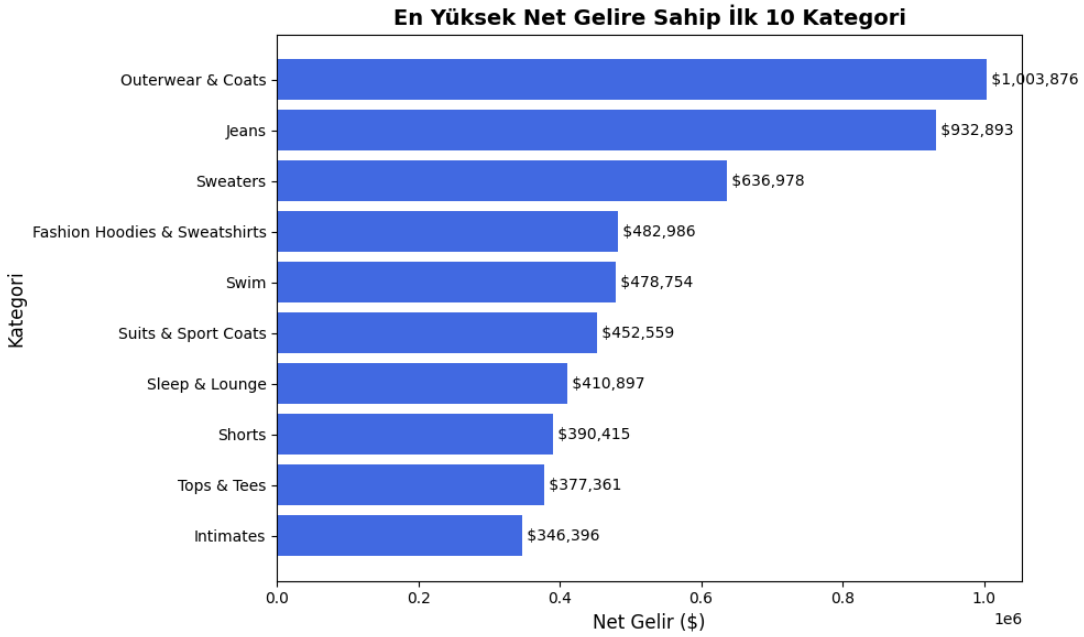
Tops & Tees, Jeans'e yakın sayıda kalem satıyor ama kalem başına fiyatı Jeans'in yarısından az. Bu yüzden net geliri de Jeans'in yarısından az. Hacim ile değer arasındaki bu fark, kampanya önceliklerini belirlerken dikkate alınmalı.

Veri tarihi notu: theLook sentetik bir veri seti ve düzenli olarak yeniden üretiliyor. Bu yüzden rakamlar, farklı tarihte hazırlanan Modül 1 raporundan biraz farklı (ör. Jeans net geliri 939 bin $ yerine 933 bin $). Rapor ve dashboard'larda veri tarihini belirtmek bu nedenle önemli.

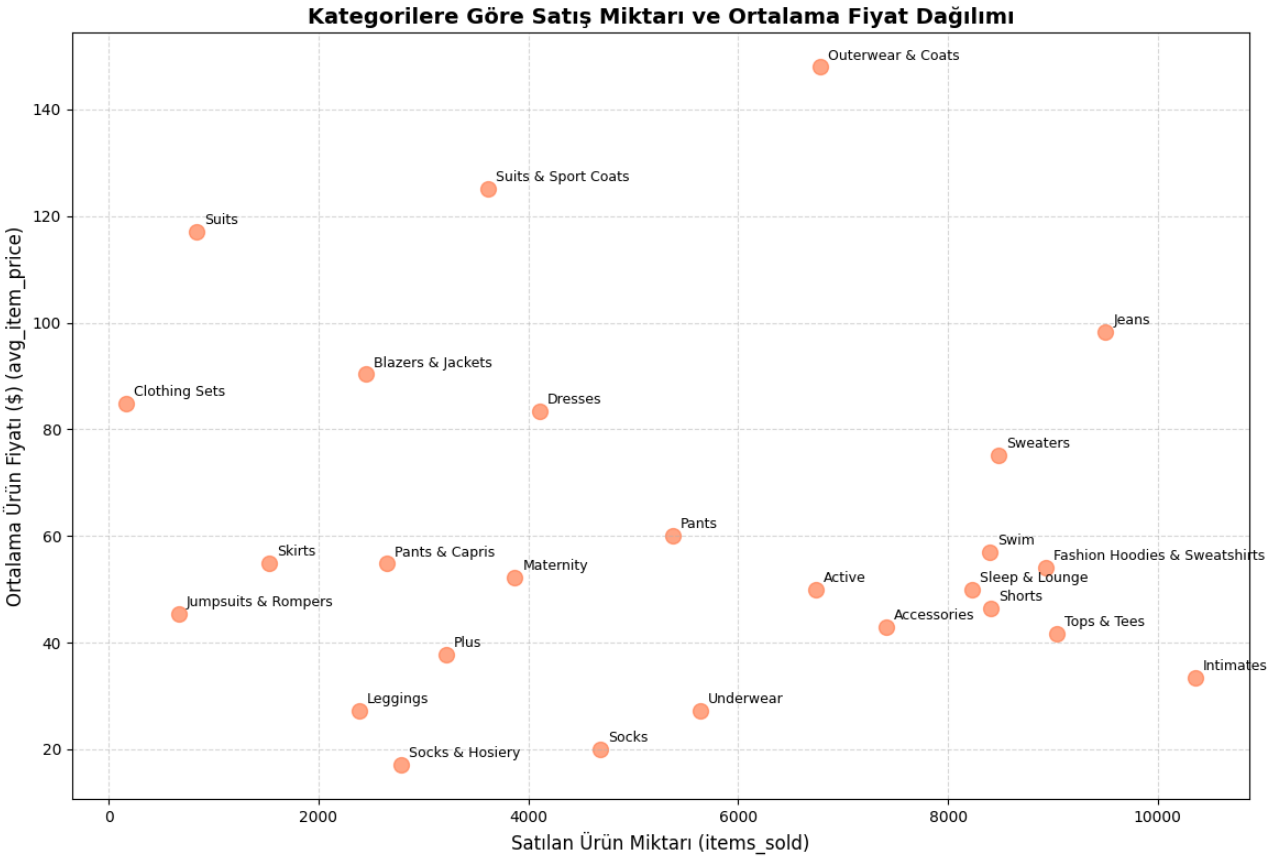
5. Python ile analiz (Google Colab)

Mart tablosu, Google Colab'da BigQuery istemcisiyle doğrudan bir pandas DataFrame'e çekildi. Böylece dbt ile hazırlanan, test edilmiş veri Python analizine kaynak oldu. DataFrame'e iki yeni metrik eklendi: **avg_item_price** (net gelir / satılan kalem) ve **loss_rate** (iptal + iade / brüt gelir).

```
client = bigquery.Client(project="bigquery-datascientist")
df = client.query(
    "select * from `bigquery-datascientist.dbt_zeynep.fct_category_revenue`"
).to_dataframe()
df["avg_item_price"] = df["net_revenue"] / df["items_sold"]
df["loss_rate"] = (df["cancelled_revenue"] + df["returned_revenue"]) / df["gross_revenue"] * 100
```



Net gelire göre ilk 10 kategori (matplotlib).



26 kategori için satılan kalem sayısı ve kalem başına ortalama fiyat (matplotlib).

Yorum: Grafik kategorileri dört gruba ayırıyor. **Çok satan ve pahalı** (sağ üst): Outerwear & Coats (~148 $) ve Jeans (~98 $). Gelirin lideri olmaları hem hacimden hem fiyattan geliyor. **Çok satan ama ucuz** (sağ alt): Intimates en yüksek satış adedine sahip ama kalem başına ~33 $ ile net gelirde ancak 10. sırada. **Az satan ama pahalı** (sol üst): Suits ve Clothing Sets. Bu kategorilerde birkaç ek satış bile gelire belirgin katkı yapar. **Az satan ve ucuz** (sol alt): Socks & Hosiery ve Leggings.

Öneri: Sağ alttaki kategorilerde (Intimates, Tops & Tees) paket ve çoklu alım kampanyaları sepet tutarını artırabilir. Sol üstteki pahalı kategorilerde ise görünürlük ve hedefli reklam, satış adedini artırmak için daha uygun bir araç.

6. Karşılaşılan sorunlar ve çözümleri

Geliştirme sırasında çıkan hatalar ve çözümleri aşağıda. Hata mesajlarını okuyup kök nedeni bulmak, sürecin önemli bir parçasıydı.

Sorun	Kök neden	Çözüm
Sipariş kalemleri tablosunda unique testleri başarısız olurdu	order_id, user_id, product_id bu tabloda tekrar eden kolonlar	unique testi sadece order_item_id'ye konuldu, diğerlerine not_null
stg_products: "Expected end of input but got identifier category"	Kolon listesinde eksik virgül	Virgül eklendi, son kolondaki fazla virgül silindi
stg_products testi: "Table not found"	Model oluşamadığı için test edilecek tablo yoktu (ilk hatanın yan etkisi)	İlk hata düzeltilince kendiliğinden çözüldü
items_sold kolonu hata verdi	IF fonksiyonunda doğru ve yanlış değerleri eksikti	COUNT(DISTINCT IF(koşul, order_item_id, NULL))
dbt deprecation uyarısı	dbt 1.12'de test argümanları arguments altında olmalı	accepted_values testleri yeni sözdizimine taşındı

7. Kullanılan teknikler

dbt Core (source, ref, staging/mart katmanları, materialization: view ve table), generic testler (unique, not_null, accepted_values), dbt build, dbt docs ve lineage, BigQuery (OAuth bağlantısı, US konumu), koşullu toplama (SUM(IF), COUNT(DISTINCT IF)), Google Cloud Shell, Git; Python: Google Colab, BigQuery istemcisi, pandas, matplotlib.

Ek A. Proje kodu

models/staging/_sources.yml

```
version: 2
sources:
  - name: thelook
    database: bigquery-public-data
    schema: thelook_ecommerce
    tables:
      - name: orders
      - name: order_items
      - name: products
```

models/staging/stg_order_items.sql

```
select
  id as order_item_id,
  order_id,
  user_id,
  product_id,
  status,
  created_at,
  shipped_at,
  delivered_at,
  returned_at,
  sale_price
from {{ source('thelook', 'order_items') }}
```

models/staging/stg_products.sql

```
select
  id as product_id,
  category,
  name,
  brand,
  cost,
  retail_price,
  department,
  sku
from {{ source('thelook', 'products') }}
```

stg_orders.sql de aynı yapıda: orders kaynağından order_id, user_id, status, tarih kolonları ve num_of_item seçiliyor.

models/staging/_staging.yml

```
version: 2
models:
  - name: stg_orders
    columns:
      - name: order_id
        tests: [unique, not_null]
      - name: status
        tests:
          - accepted_values:
              arguments:
                values: ['Processing', 'Shipped', 'Complete', 'Cancelled', 'Returned']
  - name: stg_order_items
    columns:
      - name: order_item_id
        tests: [unique, not_null]
      - name: order_id
        tests: [not_null]
      - name: product_id
        tests: [not_null]
  - name: stg_products
    columns:
      - name: product_id
        tests: [unique, not_null]
      - name: department
        tests:
          - accepted_values:
              arguments:
                values: ['Men', 'Women']
```

models/marts/fct_category_revenue.sql

```
with items as (
  select * from {{ ref('stg_order_items') }}
),
products as (
  select * from {{ ref('stg_products') }}
)
select
  products.category,
  round(sum(sale_price), 2) as gross_revenue,
  round(sum(if(status = 'Cancelled', sale_price, 0)), 2) as cancelled_revenue,
  round(sum(if(status = 'Returned', sale_price, 0)), 2) as returned_revenue,
  round(sum(if(status not in ('Cancelled', 'Returned'), sale_price, 0)), 2) as net_revenue,
  count(distinct if(status not in ('Cancelled', 'Returned'), order_item_id, null)) as items_sold
from items
join products on items.product_id = products.product_id
group by products.category
```

models/marts/_marts.yml

```
version: 2
models:
  - name: fct_category_revenue
    columns:
      - name: category
        tests: [unique, not_null]
      - name: net_revenue
        tests: [not_null]
```

dbt_project.yml (models bölümü)

```
models:
  ecommerce_warehouse:
    staging:
      +materialized: view
    marts:
      +materialized: table
```